KP18 The Probabilistic Neural Network (PNN)

> Level: undergraduate, year 3
>
> Pattern Recognition | Chapter 4, §4.17

Slides: read and download

Learning objectives

- Explain how a PNN moves the Parzen estimate into a network and how unit-norm normalization turns a distance into an inner product.
- Write the hidden and output layers of a PNN and compute the outputs and the class on a small example.
- Explain the cost and the limit of a PNN: a hidden layer as large as the training set and a normalization that discards the length.

KP18 The Probabilistic Neural Network (PNN)

One-sentence summary: A Parzen estimate is an average of kernels, so give every training point one neuron; after normalizing the vectors to length 1 the squared distance becomes an inner product and the formula takes the form of a network. Training is copying the data, and the price is keeping every training point.

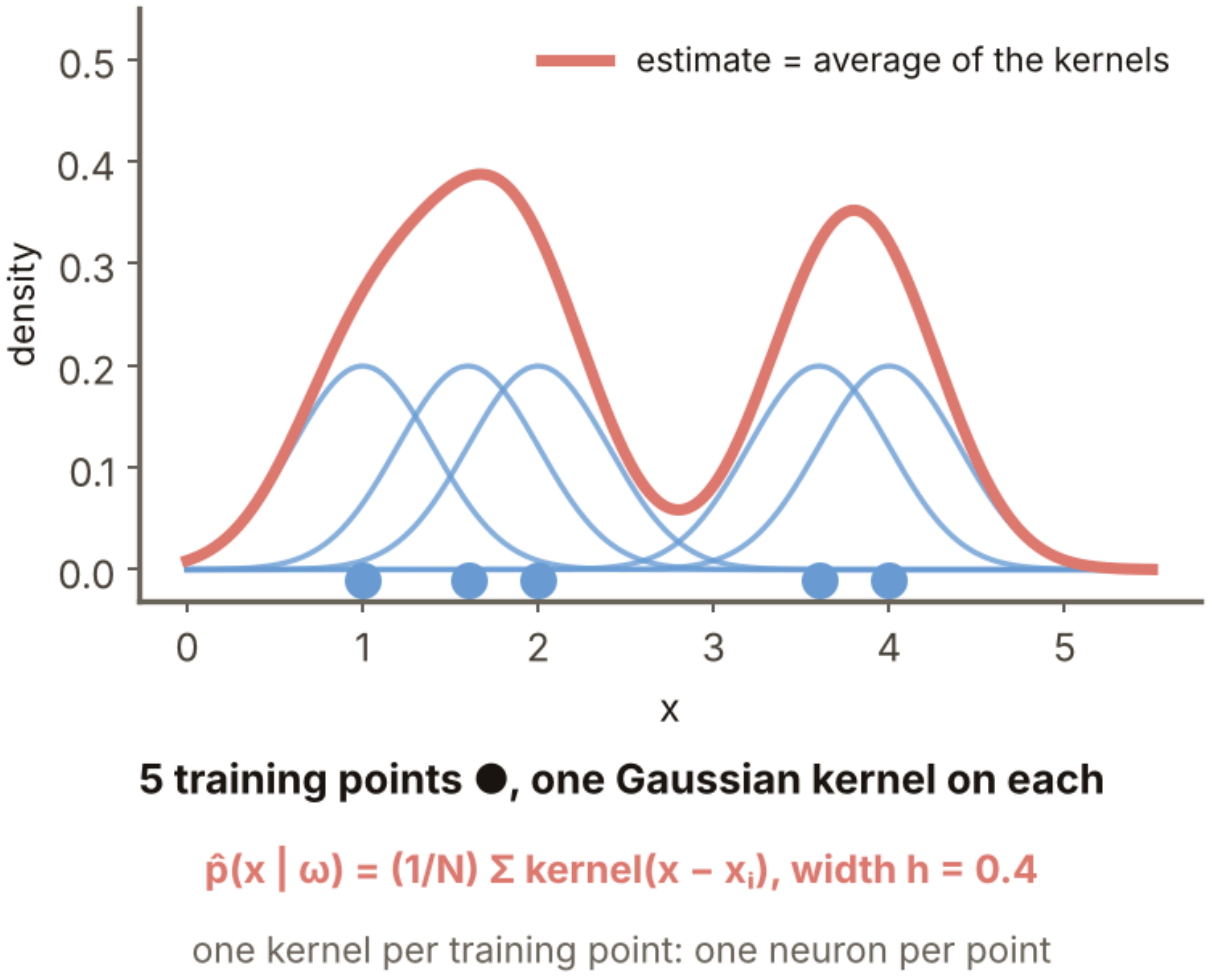
1. One neuron per point

Plain explanation: Is there a network that needs almost no training?

- **Parzen estimate**: a kernel (Gaussian here) on every training point, averaged to estimate the class density $\hat{p}(x \mid \omega)$.
- **Kernel width** h: the width of the kernel; larger is smoother and smaller hugs the individual training points.

The Bayes decision compares the largest $P(\omega_i)\,\hat{p}(x \mid \omega_i)$. A sum of kernels is easy to compute in parallel, so every training point gets one neuron.

Figure `lec05-kp18-parzen`: five one-dimensional training points with a kernel each, averaged into the estimated density.



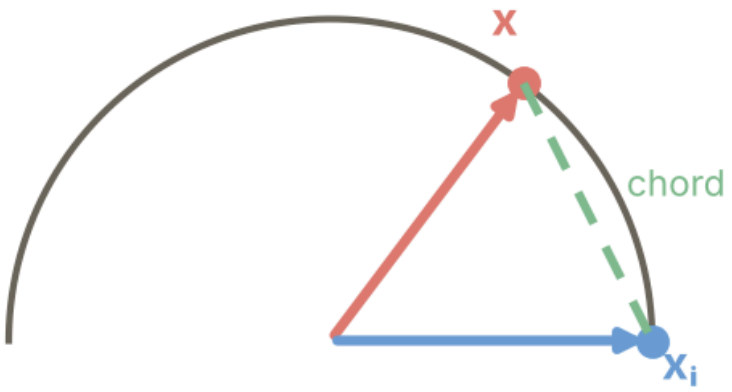
2. Distance becomes inner product

Unit-norm normalization: divide every feature vector by its own length $\|x\| = \sqrt{\sum_j x_j^2}$ so that its length becomes 1. Expanding the squared distance

$$\|x - x_i\|^2 = \|x\|^2 + \|x_i\|^2 - 2x_i^T x,$$

it equals $2 - 2x_i^T x$ when both lengths are 1. **Example**: $x = (0.6, 0.8)$, $x_i = (1, 0)$: inner product 0.6 and squared distance $2 - 1.2 = 0.8$; directly $(0.4)^2 + 0.8^2 = 0.8$, the same.

Figure `lec05-kp18-circle`: two vectors on the unit circle and the chord between them.



3. The discriminant

Formal definition: after substituting $\|x - x_i\|^2 = 2 - 2x_i^T x$ and dropping the common $1/((2\pi)^{l/2} h^l)$,

$$g(\omega_m) = \frac{P(\omega_m)}{N_m} \sum_{i=1}^{N_m} \exp\Big(\frac{x_i^T x - 1}{h^2}\Big).$$

Check ($h = 0.8$): for the unit vector $x = (0.6, 0.8)$ the distance form $\exp(-\|x - x_i\|^2 / 2h^2)$ and the inner-product form $\exp((x_i^T x - 1)/h^2)$ both give 0.5353.
Common misconception: the formula can be used without normalizing. For $x = (1.2, 1.6)$, not a unit vector, the distance form gives 0.1312 and the inner-product form 1.3668, far apart; normalization is required.

Figure `lec05-kp18-kernels`: the two forms for a unit and a non-unit vector.

	distance form	inner-product form
x = (0.6, 0.8), unit $\|x - x_i\|^2 = 0.8$	0.5353	0.5353
x = (1.2, 1.6), not unit $\|x - x_i\|^2 = 2.6$	0.1312	1.3668

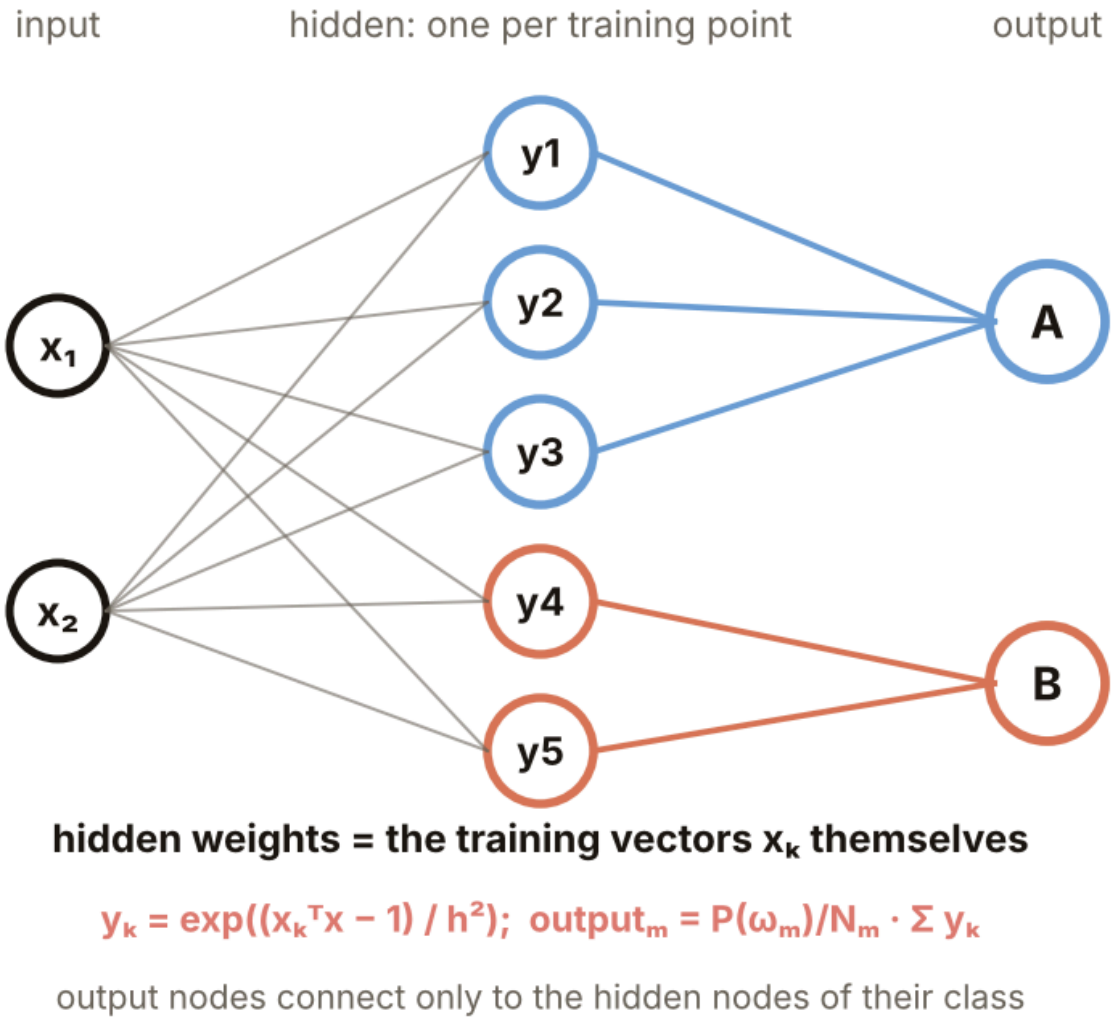
x_i = (1, 0), h = 0.8

the two forms agree only when the vectors are normalized

4. The architecture

Probabilistic neural network (PNN): the formula above moved into a network. The input layer has l nodes; the hidden layer has N nodes, the weights of node k are the k-th training vector and its output is $y_k = \exp((x_k^T x - 1)/h^2)$; the output layer has one linear combiner per class connected only to the hidden nodes of that class: $\text{output}_m = \frac{P(\omega_m)}{N_m} \sum y_k$, and the largest wins. This network needs almost no training: no gradient descent, no local minima, no epochs.

Figure `lec05-kp18-arch`: two inputs, five hidden nodes and two outputs.



5. A full worked example

Example (designed here): class A at $20^\circ, 40^\circ, 60^\circ$, class B at $100^\circ, 120^\circ$, test vector at 70°, $h = 0.5$, priors 0.5. Hidden outputs y_k: A: $0.2396, 0.5851, 0.9410$; B: $0.5851, 0.2396$. Outputs: A $= \frac{0.5}{3} \times 1.7658 = 0.2943$; B $= \frac{0.5}{2} \times 0.8247 = 0.2062$; A wins.

Figure `lec05-kp18-toy`: the five hidden outputs and the two class outputs.

test vector at 70°, h = 0.5, equal priors 0.5

training vector	class	$x_k^T x$	y_k
20°	A	0.6428	0.2396
40°	A	0.8660	0.5851
60°	A	0.9848	0.9410
100°	B	0.8660	0.5851
120°	B	0.6428	0.2396

output A = 0.5/3 · 1.7658 = 0.2943

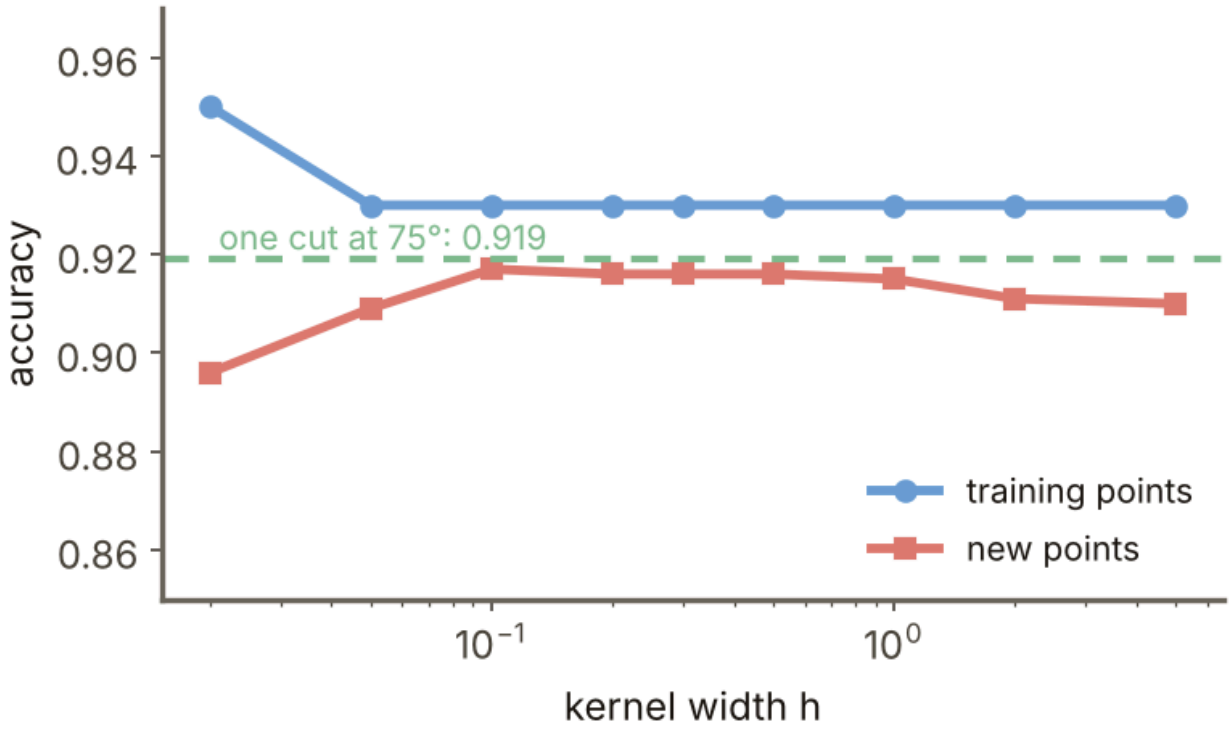
output B = 0.5/2 · 0.8247 = 0.2062

the larger output wins: class A

6. The kernel width

Example (designed here): two classes distinguished by direction (means 40° and 110°, sd 25°) with random lengths; 50 training and 500 new points per class, normalized and fed to a PNN. $h = 0.02$ gives 95% on training (the classes overlap, so never 100%) and 89.6% on new points (hugging single points); $h = 0.1 \sim 1$ gives about 91.6%; $h = 5$ still gives 91.0%. A single cut at 75° gets 91.9% on new points and the PNN is close to it; this easy example is insensitive to h and only a very small h hurts.

Figure `lec05-kp18-hcurve`: accuracy against the kernel width h.



two classes by direction, 100 training and 1000 new points

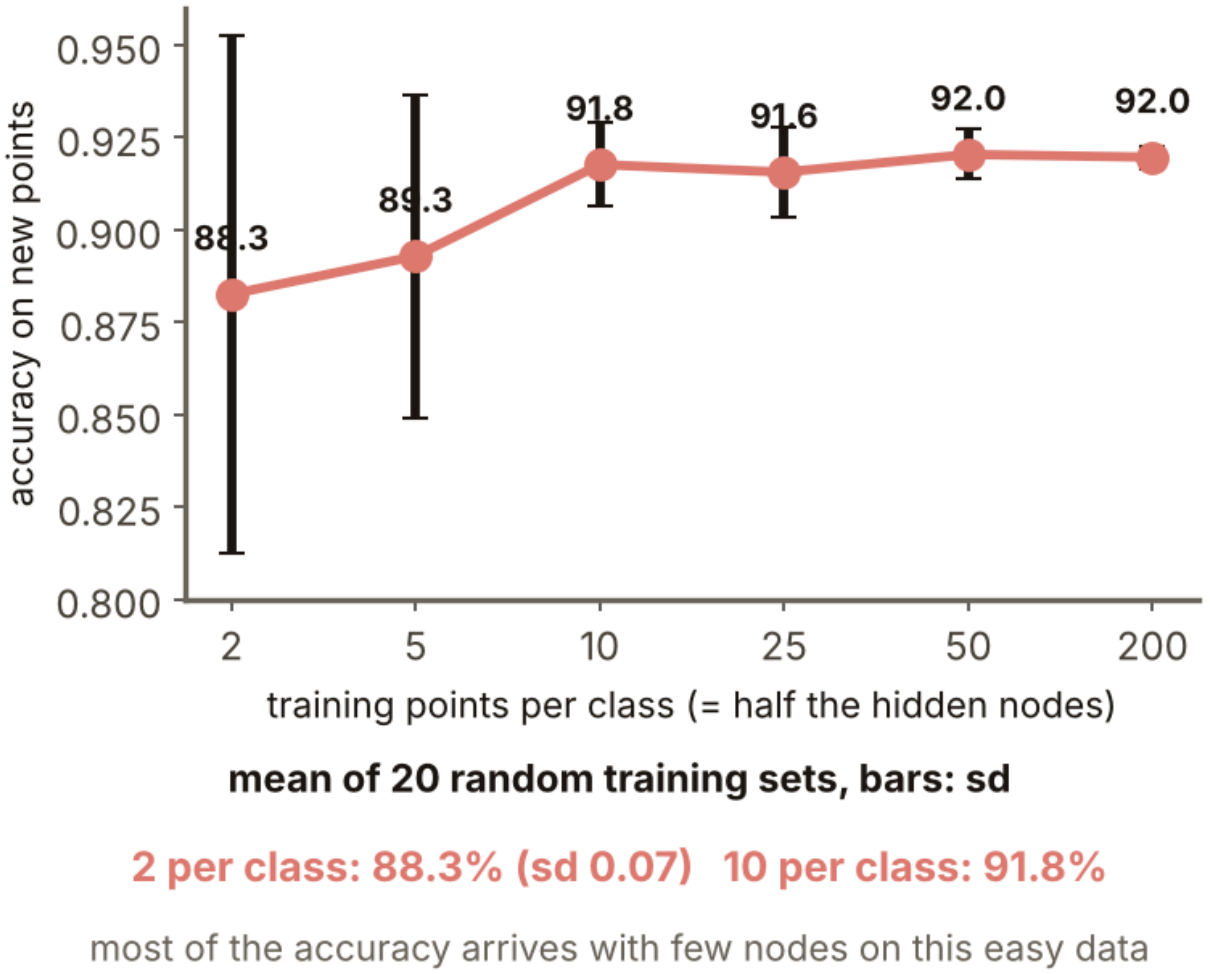
h = 0.02: 89.6%; h = 0.1 to 1: about 91.6%

only a very small h hurts; this easy data is insensitive to h

7. How many points

The same data with n training points per class, $h = 0.3$, 20 random draws each. Mean accuracy on new points: $n = 2$: 88.3% (sd 0.07); $n = 10$: 91.8% (0.011); $n = 50$: 92.0%; $n = 200$: 92.0%. This easy data need not be kept in full; fewer points mean a smaller network.

Figure `lec05-kp18-nsub`: training points per class against accuracy on new points.



8. The cost

Each test point needs $N \cdot l$ multiply-adds and $N \cdot l$ stored numbers (8 bytes each): $N = 100, l = 2$: 200 operations, 1.6 KB; $N = 10^4, l = 10$: 10^5 operations, 0.8 MB; $N = 10^6, l = 100$: 10^8 operations, 0.8 GB. **To remember**: PNN = a parallel hardware version of the Parzen estimate; training is copying data and inference computes everything, moving the learning cost to the inference cost. It sits at one end of a spectrum with sparsification (KP21): keep all against keep only the support vectors (source).

Figure `lec05-kp18-cost`: the cost table for work and memory.

points N	inputs I	work / test	memory
100	2	200	1.6 KB
10,000	10	100,000	0.8 MB
1,000,000	100	100,000,000	0.8 GB

work = N · I multiply-adds, memory = N · I numbers

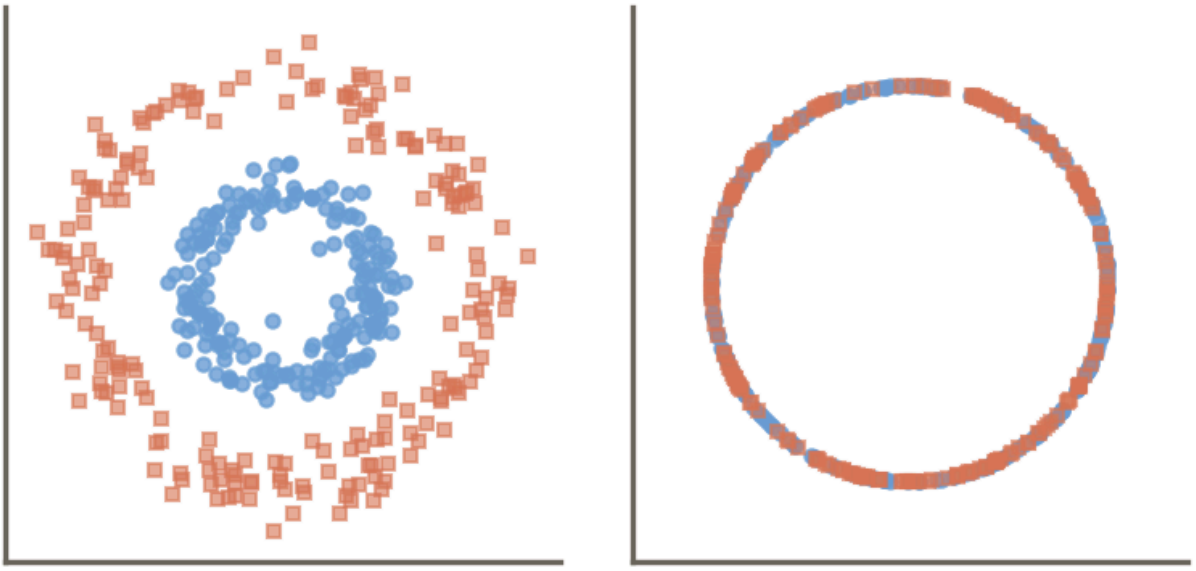
no training loop, but every training point is kept

learning cost moved to inference cost

Extension: the price of normalization, practice and worked solution

The price of normalization (designed here): concentric rings whose classes differ only in radius (about 1.0 against 2.2), 200 points each, half for training and half for testing. After normalization both classes sit on the unit circle with the same angle distribution and the PNN gets only 49.5% on new points, which is guessing. Normalization discards the length; before using a PNN check that the class information lies in the direction and not in the length.

Figure `lec05-kp18-rings`: left two concentric rings, right after normalization.



Practice: training vectors $x_A = (1, 0)$ and $x_B = (0, 1)$, test vector $x = (0.8, 0.6)$, priors 0.5; $y_k = \exp((x_k^T x - 1)/h^2)$.

1. For $h = 1$, which class?
2. For $h = 0.2$, which class, and how does the ratio of the outputs change?

Figure `lec05-kp18-practice`: the setup of the practice.

training vectors x_A = (1, 0), x_B = (0, 1)

test vector x = (0.8, 0.6), equal priors 0.5

$y_k = \exp((x_k^T x - 1) / h^2)$, output = $0.5 \cdot y$

question 1: h = 1, which class?

question 2: h = 0.2, which class, and what changes?

Worked solution of questions 1 and 2: (1) $h = 1$: $x_A^T x = 0.8$ and $x_B^T x = 0.6$; $y_A = e^{-0.2} = 0.819$ and $y_B = e^{-0.4} = 0.670$; outputs 0.409 against 0.335, class A. (2) $h = 0.2$: $y_A = e^{-5} = 0.0067$ and $y_B = e^{-10} = 0.000045$; outputs 0.0034 against 0.00002, still class A, with the ratio growing from 1.22 to about 148. (3) Check: the smaller the h the more a near training point is magnified and a far one suppressed.

Figure `lec05-kp18-practice-sol`: the calculations for the two widths.

h = 1: $x_A^T x = 0.8$, $x_B^T x = 0.6$

$y_A = e^{(-0.2)} = 0.819$, $y_B = e^{(-0.4)} = 0.670$

outputs 0.409 against 0.335

→ class A

h = 0.2: $y_A = e^{(-5)} = 0.0067$, $y_B = e^{(-10)} = 0.000045$

outputs 0.0034 against 0.00002

→ class A, ratio 148

a smaller h widens the gap between the outputs

Terms and sources

English	中文
Parzen estimate	Parzen 估計
Kernel width	核寬
Unit-norm normalization	單位範數正規化
Probabilistic neural network (PNN)	機率神經網路

Source: Chapter 4 KP18, §4.17 (read 2026-10-25). The Parzen estimate and the Bayes decision, the unit-norm simplification, the discriminant, the architecture, "almost no training" with its cost and the "to remember" line follow the source; the 70° toy, the kernel-width and training-size experiments, the cost table and the failure on concentric rings were designed and recomputed for this lecture ([added], (program)).